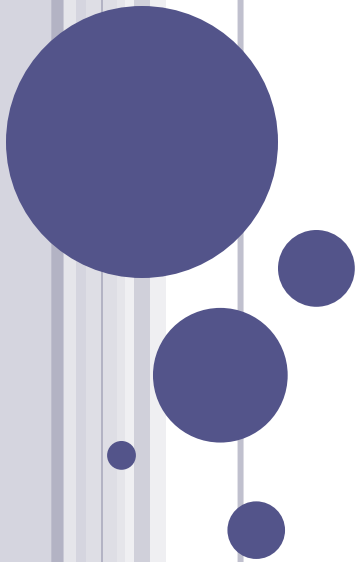


CHAPITRE2:

CONCEPTION DES BD RÉPARTIES



SCHÉMAS D'UNE BD RÉPARTIE

- Dans un contexte centralisé :

Analyse des besoins



Modèle conceptuel de données



Modèle logique

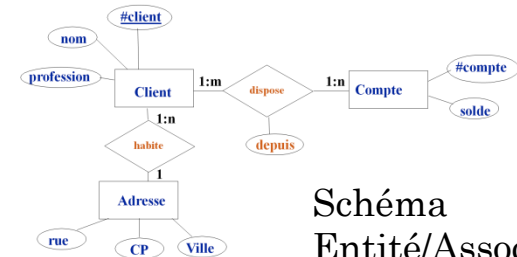


Schéma
Entité/Association

Modèle relationnel

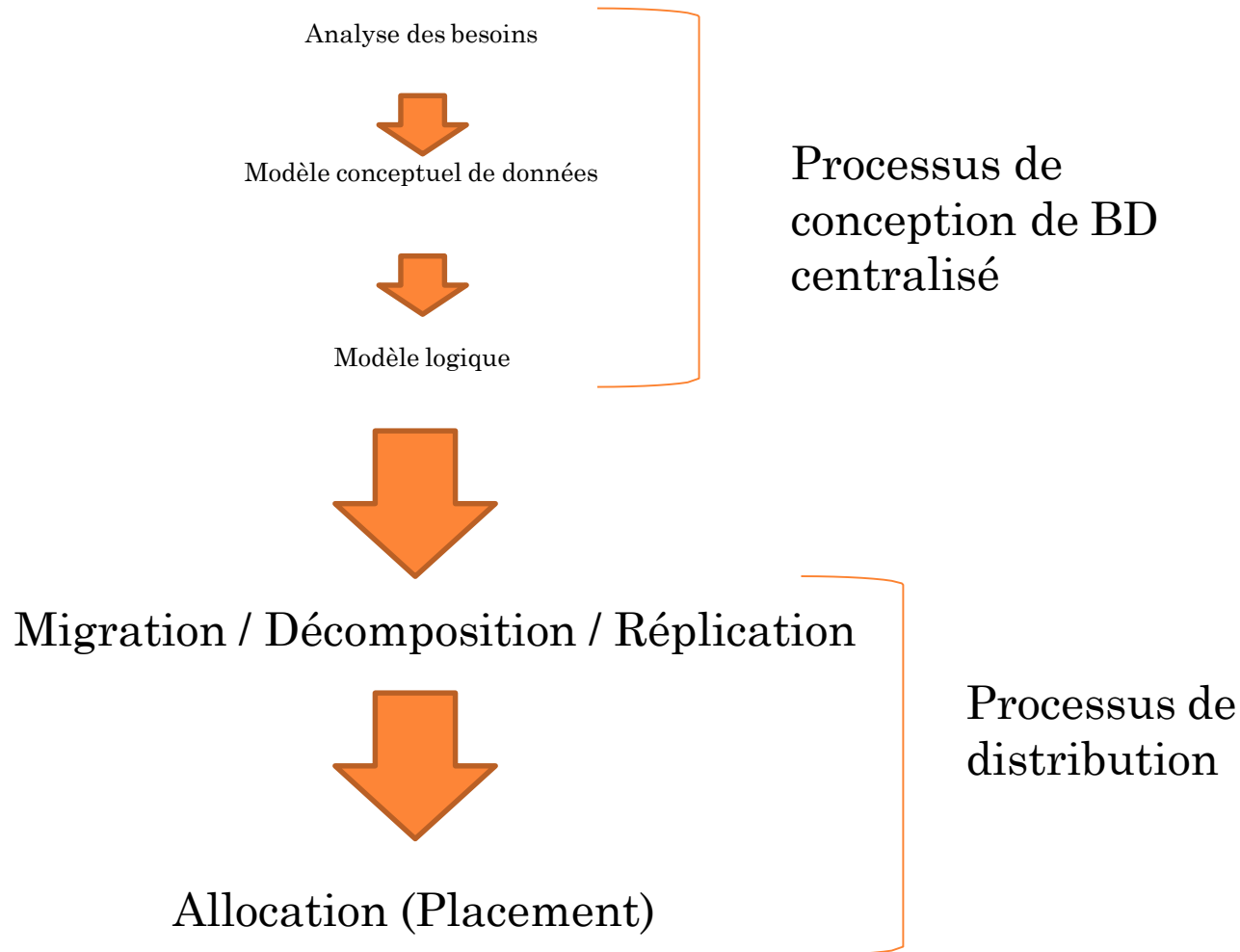
Acteur		
Nom	Prénom	Nationalité
Cruz	Penelope	ESP
Maura	Carmen	ESP
Johanson	Scarlett	USA
Villeret	Jacques	FR
Lhermite	Thierry	FR
Downey	Robert	UK
Allen	Woody	USA

Réalisateur		
Nom	Prénom	Pays
Almodovar	Pedro	ESP
Allen	Woody	USA
Veber	Francis	FR
Favreau	John	USA

Artiste = Acteur $\cup$ Réalisateur

Nom	Prénom	Nationalité
Cruz	Penelope	ESP
Maura	Carmen	ESP
Johanson	Scarlett	USA
Villeret	Jacques	FR
Lhermite	Thierry	FR
Downey	Robert	UK
Almodovar	Pedro	ESP
Allen	Woody	USA
Veber	Francis	FR
Favreau	John	USA

CONTEXTE DÉCENTRALISÉ

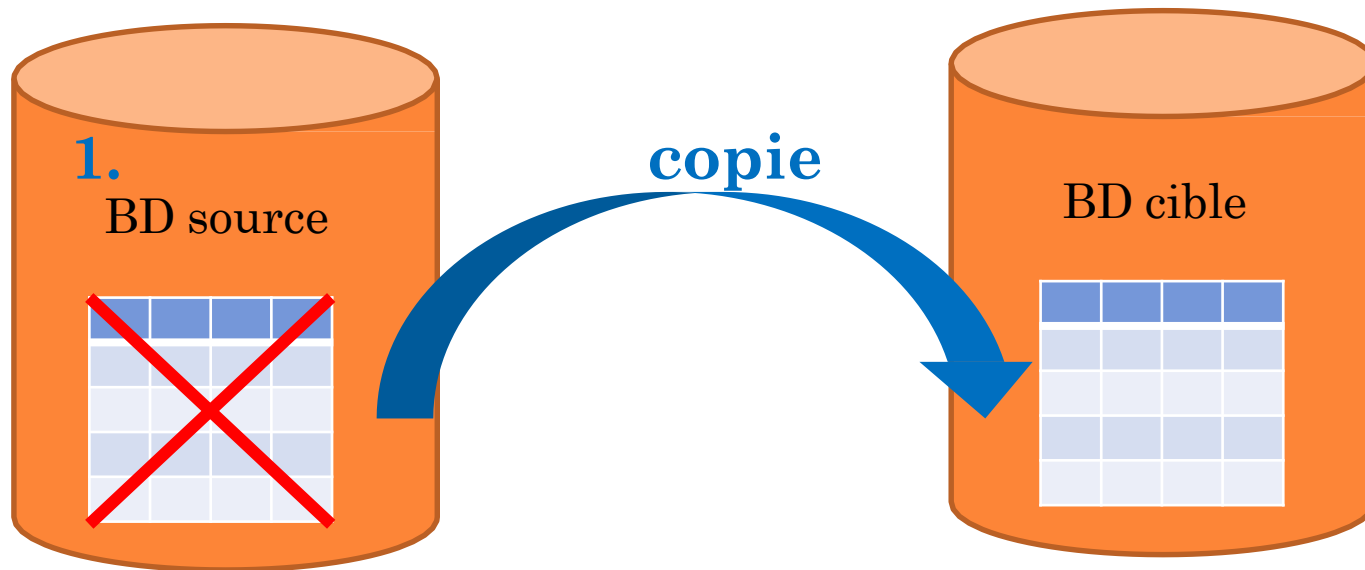


CONTEXTE DÉCENTRALISÉ

- Le schéma global est constitué :
 - d'un schéma conceptuel global
 - Contenant la description globale et unifiée de toutes les données de la BDR
 - Indépendant de la répartition des données
 - d'un schéma de placement (d'allocation)
 - Contenant les règles de correspondance avec les données locales

MIGRATION

- Transfert d'une relation complète sur un site distant



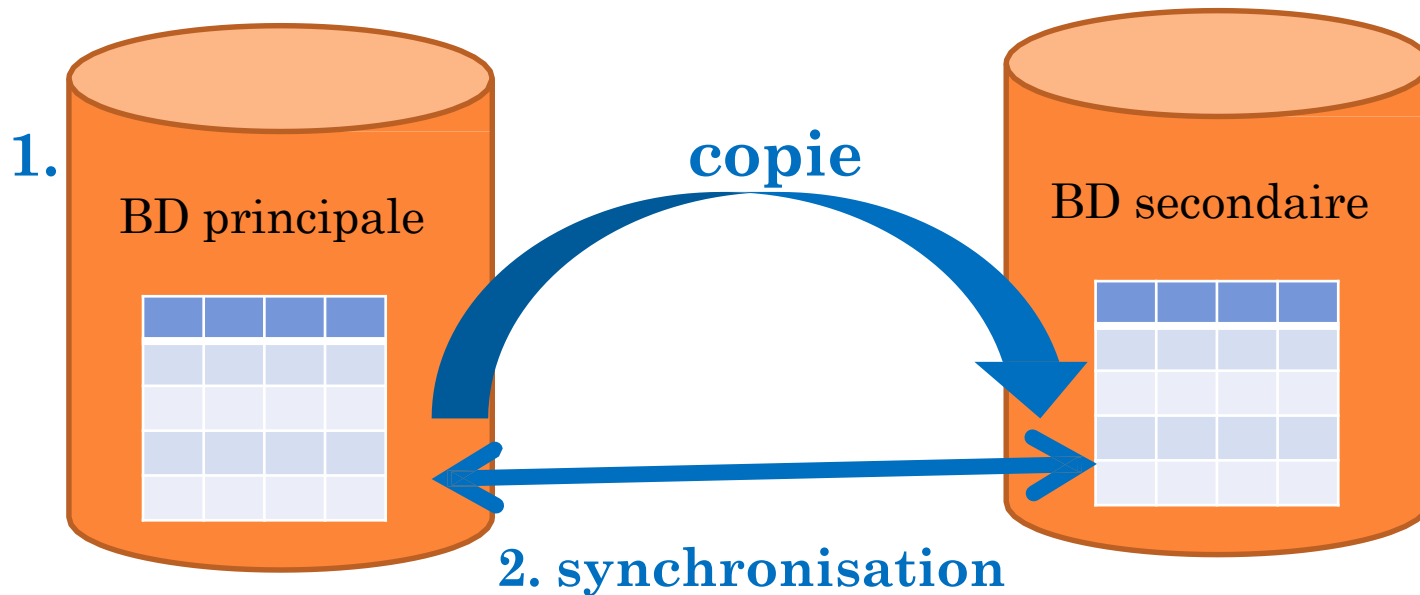
2.suppression

- Intérêt
 - Rapprocher les données des besoins



RÉPLICATION/DUPLICATION

- Création d'une copie conforme d'une table (ou ensemble de tuples) sur un site distant. La copie doit rester cohérente avec les données sources

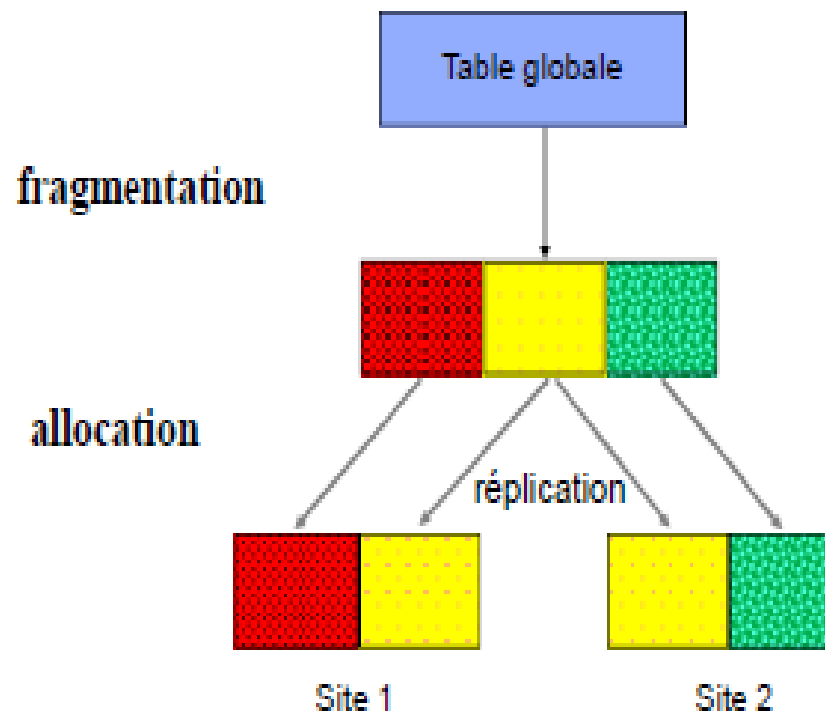


- **Intérêt**

- Avoir les mêmes données sur différents sites

FRAGMENTATION/DÉCOMPOSITION

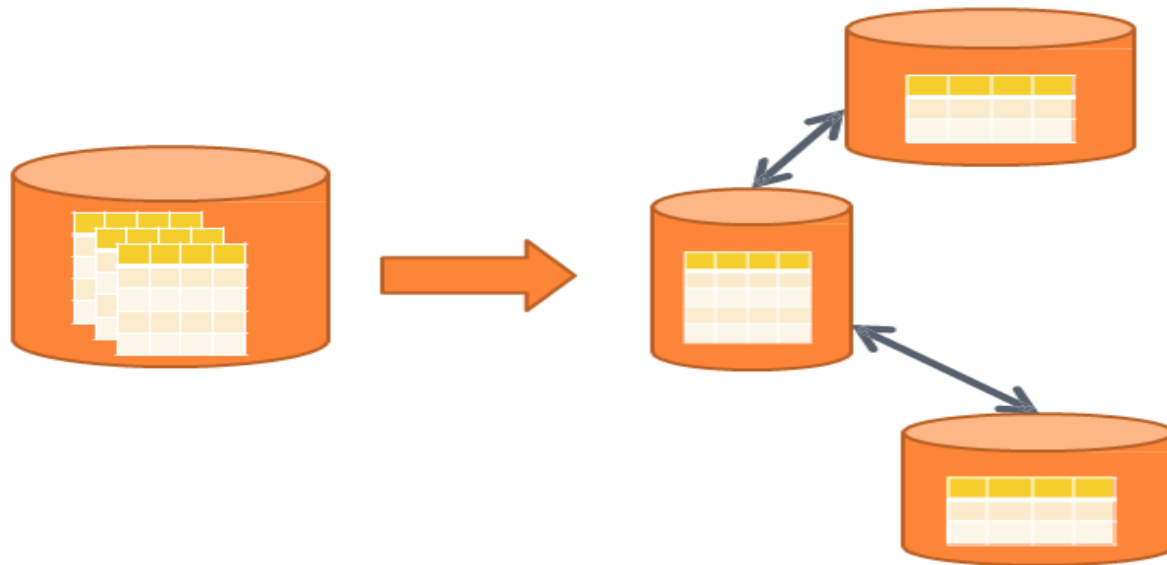
Conception par décomposition



○ Fragmentation

DÉFINITION DE LA FRAGMENTATION

- .La fragmentation est le processus de décomposition d'une base de donnée logique en un ensemble de "sous" bases de données. Cette décomposition doit être sans perte d'information.



LES RÈGLES DE FRAGMENTATION

- **Les règles à appliquer sont :**
- 1. La complétude : pour toute donnée d'une relation R , il existe un fragment R^i de la relation R qui possède cette donnée.
- 2. La reconstruction : pour toute relation décomposée en un ensemble de fragments R_i , il existe une opération de reconstruction.
- 3. La Disjonction: assure que les fragments d'une relation sont disjoints deux à deux. chaque élément de R (hormis les clés) ne doit pas être dupliqué

INTERET DE LA FRAGMENTATION

- **Exemple**

- Relation Employé

E(enum, nom, site, salaire,...)

- Un employé n'appartient qu'à un seul site

- 40 000 employés à Paris, 40 000 à Lyon, et 20 000 à Marseille

- Charge de travail (workload)

- 90% des requêtes à Paris/Lyon/Marseille sont sur des employés locaux

INTÉRÊT DE LA FRAGMENTATION

- Pour illustrer
- Accès disque = 1 unité & Accès réseau = 10 unités
- **Sans fragmentation** (un SGBD à Paris)

Coût

- à Paris = 1 unité
- à L/M = 10
- moyenne = $0.4 * 1 * (0.9 + 0.01) + 0.6 * 10 * (0.9 + 0.01) = 6.4$ unités

- **Avec fragmentation** (3 SGBD : BD-P, BD-L, BD-M)

Coût

- – à P/L/M = $0.9 * (0.4 + 0.4 + 0.2) * 1 + 0.1 * 10 = 1.9$
 - ↻ moyenne = 1.9 unités
 - ↻ Evite le risque de saturer le SGBD de Paris

TECHNIQUES DE FRAGMENTATION

- Il existe plusieurs techniques de fragmentation:
 - Fragmentation horizontale
 - Primaire
 - Dérivée
 - Fragmentation verticale
 - Fragmentation mixte

FRAGMENTATION HORIZONTALE

- **Fragmentation horizontale primaire :**
- La Fragmentation horizontale est définie par l'opération de sélection
- **Exemple**
 - Client (NCL, Nom, Ville) peut être fragmentée :
 - Client1= SELECT * FROM Client WHERE Ville = "Paris"
 - Client2= SELECT * FROM Client WHERE Ville <> "Paris"
- **Reconstruction de la relation initiale:**
- **Client = Client1 $\cup$ Client2**

Client	NCL	Nom	Ville
	C1	Dupont	Paris
	C2	Durant	Lille
	C3	Martin	Nice
	C4	Olivier	Paris

Client1	NCL	Nom	Ville
	C1	Dupont	Paris
	C4	Olivier	Paris

Client2	NCL	Nom	Ville
	C2	Durant	Lille
	C3	Martin	Nice

FRAGMENTATION HORIZONTALE

- Fragmentation horizontale dérivée :
- La Fragmentation d'une table en fonction des fragments horizontaux d'une autre table.
- Exemple
- Commande (NCL, N°Produit, Date, Qte, N°Représentant)

Commande1= SELECT * FROM Commande

WHERE NCL IN

(SELECT NCL FROM CLIENT1)

Commande2= SELECT * FROM Commande

WHERE NCL IN

(SELECT NCL FROM CLIENT2)

Cmd	NP	NCL	Date	Qte
	P1	C1	1/1/99	10
	P2	C2	1/1/99	20
	P2	C2	2/1/99	30
	P3	C4	4/1/99	30

Cmd1	NP	NCL	Date	Qte
	P1	C1	1/1/99	10
	P3	C4	4/1/99	30

Cmd2	NP	NCL	Date	Qte
	P2	C2	1/1/99	20
	P2	C2	2/1/99	30

Reconstruction de la relation initiale:
Commande = Commande1 $\cup$ Commande2

FRAGMENTATION VERTICALE

- La fragmentation verticale est le découpage d'une table en sous-tables par des projections permettant de sélectionner les colonnes qui composent chaque fragment → Répartition des attributs
- La fragmentation verticale est obtenue par décomposition de la table en groupes de colonnes.
- Toutes les valeurs des occurrences pour un même attribut se trouvent dans le même fragment
- L'opérateur de partitionnement est la projection (Π)
- L'opérateur de recomposition est la jointure ($\bowtie$)

FRAGMENTATION VERTICALE

○ Exemple

- Soit le partitionnement de la relation *Client* en deux relations :

Relation Client

No client	Nom client	Prénom	Age
174723	Villard	Jean	29
177498	Cattell	Blaise	38
201639	Tsellis	Alan	51
203446	Kowalsky	Valdimir	36

- $Client1 = \pi[NoClient, NomClient] Client$
- et $Client2 = \pi[Noclient, Prénom, Age] Client$

Client (N°Client, Nom, Prénom, Age) peut être fragmentée :

Client1= SELECT N°Client, Nom FROM Client

Client2= SELECT N°Client, Prénom, Age FROM Client

NoClient	NomClient
174 723	Villard
177 498	Cattell
201 639	Tsellis
203 446	Kowalsky

Relation Cli1

NoClient	Prénom	Age
174 723	Jean	29
177 498	Blaise	38
201 639	Alan	51
203 446	Vladimir	36

Relation Cli2

La relation d'origine est obtenue avec la recombinaison suivante :
 $Client = Cli1 \bowtie Cli2$

FRAGMENTATION VERTICALE

Remarque importante :

« Il est important de conserver les clés dans chaque fragment vertical »

Sinon reconstruction impossible !

Cde

ncde	nclient	produit	qté
D 1	C 1	P 1	10
D 2	C 1	P 2	20
D 3	C 2	P 3	5
D 4	C 4	P 4	10

Cde1

ncde	nclient
D 1	C 1
D 2	C 1
D 3	C 2
D 4	C 4

Cde2

produit	qté
P 1	10
P 2	20
P 3	5
P 4	10

FRAGMENTATION HYBRIDE

- C'est la combinaison des deux fragmentations précédentes, horizontale et verticale.
- Les occurrences et les attributs peuvent donc être répartis dans des partitions différentes.
- L'opération de partitionnement est une combinaison de projections et de sélections.
- L'opération de recomposition est une combinaison de jointures et d'unions.

FRAGMENTATION MIXTE

Fragments définis par
sélection et projection

Client

nclient	nom	ville
C 1	Dupont	Paris
C 2	Martin	Lyon
C 3	Martin	Paris
C 4	Smith	Lille

Fragmentation horizontale

Client1

nclient	nom	ville
C 1	Dupont	Paris
C 3	Martin	Paris

Client2

nclient	nom	ville
C 2	Martin	Lyon
C 4	Smith	Lille

Fragmentation verticale

Client11

nclient	nom
C 1	Dupont
C 3	Martin

Client12

nclient	ville
C 1	Paris
C 3	Paris

Fragmentation verticale

Client21

nclient	nom
C 2	Martin
C 4	Smith

Client22

nclient	ville
C 2	Lyon
C 4	Lille

FRAGMENTATION MIXTE

- Relation $Cli11 \pi[NoClient, Nom] (\sigma[Ville=Paris] Client)$
- Relation $Cli12 \pi[NoClient, Ville] (\sigma[Ville=Paris] Client)$
- Relation $Cli21 \pi[NoClient, Nom] (\sigma[Ville \neq Paris] Client)$
- Relation $Cli22 \pi[NoClient, Ville] (\sigma[Ville \neq Paris] Client)$

- La relation $Client$ est obtenue avec : $(Cli11 \bowtie Cli12) \cup (Cli21 \bowtie Cli22)$

DÉFINITION DES FRAGMENTS

- Le principe est de baser la fragmentation sur l'ensemble des requêtes d'interrogation ou de mise à jour prédéfinies.
- Il faut extraire de ces requêtes toutes les conditions de sélections, les attributs projetés et les jointures.
- Les opérations de sélection sont à la base des fragmentations horizontales, les opérations de projection sont à la base des fragmentations verticales.
- Pour éviter le risque d'émietter la base de données, on peut restreindre le nombre de requêtes prises en considération.
 - On ne s'intéresse alors qu'aux requêtes les plus fréquentes ou les plus sensibles.

DÉFINITION DES FRAGMENTS

- Approche orientée données
 - Selon un ensemble d'hypothèses :
 - Informations sur les contraintes de placement
- Approche orientée requêtes :
 - Informations sur les requêtes généralement posées

FRAGMENTATION ORIENTÉE DONNÉES

○ Contexte

On considère une base de données comprenant la table:

- **ETUDIANT** (numE, nomE, pnomE, numC)
- **VACATAIRE** (numV, nomV, pnomV)
- **CENTRE** (numC, villeC, adrC, telC, zoneC)
- **FORMATION**(numC, numF, nomF, nbHeure, numV, coutF, rentabilitéF)

avec : zoneC $\in$ {'Nord', 'Est', 'Sud', 'Ouest'}

Sites géographiques : 'Paris', 'Lyon', 'Marseille', 'Bordeaux'

○ Hypothèses

- **H1** : Paris gère les centres du nord de la France, Lyon les centres de la zone Est, Marseille les centres de zone sud et Bordeaux les centres de la zone Ouest.
- **H2** : Chaque centre gère ses étudiants
- **H3** : Les vacataires délivrent des formations dans tous les centres
- **H4** : Les informations sur les formations sont gérées à Paris, hormis les informations liées au coût et à la rentabilité qui sont stockées à Lyon

FRAGMENTATION ORIENTÉE DONNÉE

CENTRE (numC, villeC, adrC, telC, zoneC)

avec : zoneC $\in$ {'Nord', 'Est', 'Sud', 'Ouest'}

H1 : Paris gère les centres du nord de la France, Lyon les centres de la zone Est, Marseille les centres de zone sud et Bordeaux les centres de la zone Ouest.

FRAGMENTATION HORIZONTALE de CENTRE

Centre_{Paris} = $\sigma_{\text{zoneC} = \text{'Nord'}}(\text{Centre})$

Centre_{Lyon} = $\sigma_{\text{zoneC} = \text{'Est'}}(\text{Centre})$

Centre_{Marseille} = $\sigma_{\text{zoneC} = \text{'Sud'}}(\text{Centre})$

Centre_{Bordeaux} = $\sigma_{\text{zoneC} = \text{'Ouest'}}(\text{Centre})$

EXEMPLE DE FRAGMENTATION ORIENTÉE DONNÉES

ETUDIANT (numE, nomE, pnomE, numC)

H2 : Chaque centre gère ses étudiants

FRAGMENTATION HORIZONTALE de ETUDIANT DERIVEE de CENTRE

$\text{Etudiant}_{\text{Paris}} = \text{Etudiant} \begin{array}{c} \diagup \quad \diagdown \\ \text{numC} \end{array} \text{Centre}_{\text{Paris}}$

$\text{Etudiant}_{\text{Lyon}} = \text{Etudiant} \begin{array}{c} \diagup \quad \diagdown \\ \text{numC} \end{array} \text{Centre}_{\text{Lyon}}$

$\text{Etudiant}_{\text{Marseille}} = \text{Etudiant} \begin{array}{c} \diagup \quad \diagdown \\ \text{numC} \end{array} \text{Centre}_{\text{Marseille}}$

$\text{Etudiant}_{\text{Bordeaux}} = \text{Etudiant} \begin{array}{c} \diagup \quad \diagdown \\ \text{numC} \end{array} \text{Centre}_{\text{Bordeaux}}$

EXEMPLE DE FRAGMENTATION ORIENTÉE DONNÉE

VACATAIRE (numV, nomV, pnomV)

H3 : Les vacataires délivrent des formations dans tous les centres

REPLICATION de VACATAIRE

Vacataire_{Paris} = Vacataire

Vacataire_{Lyon} = Vacataire

Vacataire_{Marseille} = Vacataire

Vacataire_{Bordeaux} = Vacataire

EXEMPLE DE FRAGMENTATION ORIENTÉE DONNÉE

- **FORMATION**(numC, numF, nomF, nbHeure, numV, coutF, rentabilitéF)
- **H4** : Les informations sur les formations sont gérées à Paris, hormis les informations liées au coût et à la rentabilité qui sont stockées à Lyon

- **FRAGMENTATION VERTICALE de FORMATION**

$$\text{Formation}_{\text{Paris}} = \prod_{\text{numC}, \text{numF}, \text{nomF}, \text{nbHeure}, \text{numV}} (\text{Formation})$$

$$\text{Formation}_{\text{Lyon}} = \prod_{\text{numC}, \text{numF}, \text{coutF}, \text{rentabilitéF}} (\text{Formation})$$

$$\text{Formation}_{\text{Marseille}} = \emptyset$$

$$\text{Formation}_{\text{Bordeaux}} = \emptyset$$

FRAGMENTATION ORIENTÉE REQUÊTE

- **Fragmentation Horizontale**
- *Exploitation des clauses WHERE des requêtes*
- $\Rightarrow$ Informations sur la base de données : Avoir les prédicats simples
- **Définition :**
- Étant donnée une relation $R(A1, A2, \dots, An)$
- Un prédicat simple p_j défini sur R est défini:
- $p_j : A_i \theta \text{ valeur}$ avec: $\theta \in \{=, <, >, \leq, \geq, \neq\}$; et $\text{valeur} \in \text{domaine}(D_i)$
- Soit $Pr = \{p_1, p_2, \dots, p_m\}$ un ensemble de prédicats simples définis sur la relation R_i , l'ensemble de minterms $M = \{m_1, m_2, \dots, m_z\}$ est défini comme suit:
- $M = \{m_i \mid m_i = \bigwedge p_j \in Pr \ p_j^*, 1 \leq i \leq z, 1 \leq j \leq m;$
- where $p_j^* = p_j$ or $\neg p_j$
- Pour appliquer cet algorithme, il faut suivre les étapes suivantes :
 - **Génération des minterms**
 - **élimination des minterms contradictoires**

DÉFINITION DES FRAGMENTS

- **Exemple 1**

- p1: Ville = "Tunis"

- p2: Salaire $\leq 10\ 000$

- **Exemple:**

- $m1 : (Ville = "Tunis ") \wedge (Salaire \leq 10\ 000)$

- $m2 : NOT(Ville = "Tunis ") \wedge (Salaire \leq 10\ 000)$

- $m3 : (Ville = "Tunis ") \wedge NOT(Salaire \leq 10\ 000)$

- $m4 : NOT(Ville = "Tunis ") \wedge NOT(Salaire \leq 10\ 000)$

DÉFINITION DES FRAGMENTS

- **Exemple 2**
- Considérons les prédicats simples suivants:
- $A < 10, A > 5, \text{Loc} = \text{SA}, \text{Loc} = \text{SB}$
- Les minterms à générer sont :
- (1) $A < 10 \wedge A > 5 \wedge \text{Loc} = \text{SA} \wedge \text{Loc} = \text{SB}$
- (2) $A < 10 \wedge A > 5 \wedge \text{Loc} = \text{SA} \wedge \neg(\text{Loc} = \text{SB})$
- (3) $A < 10 \wedge A > 5 \wedge \neg(\text{Loc} = \text{SA}) \wedge \text{Loc} = \text{SB}$
- (4) $A < 10 \wedge A > 5 \wedge \neg(\text{Loc} = \text{SA}) \wedge \neg(\text{Loc} = \text{SB})$
- (5) $A < 10 \wedge \neg(A > 5) \wedge \text{Loc} = \text{SA} \wedge \text{Loc} = \text{SB}$
- (6) $A < 10 \wedge \neg(A > 5) \wedge \text{Loc} = \text{SA} \wedge \neg(\text{Loc} = \text{SB})$
- (7) $A < 10 \wedge \neg(A > 5) \wedge \neg(\text{Loc} = \text{SA}) \wedge \text{Loc} = \text{SB}$
- (8) $A < 10 \wedge \neg(A > 5) \wedge \neg(\text{Loc} = \text{SA}) \wedge \neg(\text{Loc} = \text{SB})$
- (9) $\neg(A < 10) \wedge A > 5 \wedge \text{Loc} = \text{SA} \wedge \text{Loc} = \text{SB}$
- (10) $\neg(A < 10) \wedge A > 5 \wedge \text{Loc} = \text{SA} \wedge \neg(\text{Loc} = \text{SB})$
- (11) $\neg(A < 10) \wedge A > 5 \wedge \neg(\text{Loc} = \text{SA}) \wedge \text{Loc} = \text{SB}$
- (12) $\neg(A < 10) \wedge A > 5 \wedge \neg(\text{Loc} = \text{SA}) \wedge \neg(\text{Loc} = \text{SB})$
- (13) $\neg(A < 10) \wedge \neg(A > 5) \wedge \text{Loc} = \text{SA} \wedge \text{Loc} = \text{SB}$
- (14) $\neg(A < 10) \wedge \neg(A > 5) \wedge \text{Loc} = \text{SA} \wedge \neg(\text{Loc} = \text{SB})$
- (15) $\neg(A < 10) \wedge \neg(A > 5) \wedge \neg(\text{Loc} = \text{SA}) \wedge \text{Loc} = \text{SB}$
- (16) $\neg(A < 10) \wedge \neg(A > 5) \wedge \neg(\text{Loc} = \text{SA}) \wedge \neg(\text{Loc} = \text{SB})$

Après éliminations des fragments contradictoires, **les fragments finaux sont :**

F2: $5 < A < 10 \wedge \text{Loc} = \text{SA}$

F3: $5 < A < 10 \wedge \text{Loc} = \text{SB}$

F6: $A \leq 5 \wedge \text{Loc} = \text{SA}$

F7: $A \leq 5 \wedge \text{Loc} = \text{SB}$

F10: $A \geq 10 \wedge \text{Loc} = \text{SA}$

F11: $A \geq 10 \wedge \text{Loc} = \text{SB}$

FRAGMENTATION ORIENTÉE REQUÊTES

Fragmentation verticale

- *Exploitation des clauses SELECT des requêtes*
- On se base les requêtes d'interrogation ou de mise à jour prédéfinies les plus importantes et les plus fréquentes pour extraire les attributs concernés
- Les fragments verticaux sont exclusifs, sauf en ce qui concerne le (ou les) attribut(s) de jointure (clé, identifiant) qui sont communs à tous les fragments et qui seront utilisés pour que la décomposition soit sans perte d'information (reconstruction).

FRAGMENTATION ORIENTÉE REQUÊTES

- Pour le cas de la fragmentation verticale on se base sur les conditions de projections.
 - Soient $p_1, p_2, \dots, p_n$ les conditions de projection qui ont été extraites des requêtes,
 - On forme l'ensemble des 2^n intersections des expressions de projection où chacune est soit l'expression initiale soit son complément.

$$IP = \left\{ \bigcap_{j=1..n} P_j^* \text{ où } P_j^* \text{ est soit } p_j \text{ soit } \neg p_j \right\}$$

- ➔ On ôte de cet ensemble les conjonctions de condition qui sont toujours fausses, et on simplifie les autres.

NB: Dans le cas où la requête ne possède pas d'attributs de jointure il faut les ajouter.

FRAGMENTATION ORIENTÉE REQUÊTES

Exemple :

On considère une table CUISINIER que l'on se propose de répartir en fragments verticaux

Cuisinier

NUMERO	NOM	PRENOM	NUMEQ
12	DUPONT	Jean	1
34	DUPONT	Jeanne	2
17	DUBOIS	Robert	1
22	DUBALAI	Aline	1
26	DUGENOU	Jean	2
11	DURAND	Aline	2
38	DURACUIRE	ROBERT	2
9	DURALUMIN	Roberte	1
13	DURDUR	Jean	2
20	DURALEX	Jean	1

On commencera par se baser sur les requêtes les plus fréquentes:

R1: SELECT numero, numeq FROM CUISINIER WHERE prenom = 'Jean' AND nom LIKE '%R%';

R2: SELECT * FROM CUISINIER WHERE numeq = '1';

R3: SELECT numero, nom FROM CUISINIER WHERE numeq = '2' AND prenom = 'Jean';

FRAGMENTATION ORIENTÉE REQUÊTES

Exemple (suite):

Pour effectuer la fragmentation verticale on se base sur les projections utilisées:

- P1 : (numero, numeq)
- P3 : (numero, nom)
- P2 n'est pas considérée comme une projection car tous les attributs sont demandés dans la requête

A partir des projection P_i , on construit l'ensemble IP des intersections de projections :

$$IP = \left\{ \bigcap_{j=1..n} P_j^* \text{ où } P_j^* \text{ est soit } p_j \text{ soit } \neg p_j \right\}$$

Où $\neg P_j$ désigne le complément de P_j sur l'ensemble des attributs de la table CUISINIER avec l'ajout obligatoire de la clé numero :

- $\neg P1 = (\text{numero}, \text{nom}, \text{prenom})$
- $\neg P3 = (\text{numero}, \text{prenom}, \text{numeq})$

➔ $IP = \{ P1 \cap P3, \neg P1 \cap P3, P1 \cap \neg P3, \neg P1 \cap \neg P3 \}$

FRAGMENTATION ORIENTÉE REQUÊTES

❖ Exemple (suite):

- ❖ $P1 \cap P3 = (\text{numero}, \text{numeq}) \cap (\text{numero}, \text{nom}) = (\text{numero})$
- ❖ $\neg P1 \cap P3 = (\text{numero}, \text{nom}, \text{prenom}) \cap (\text{numero}, \text{nom}) = (\text{numero}, \text{nom})$
- ❖ $P1 \cap \neg P3 = (\text{numero}, \text{numeq}) \cap (\text{numero}, \text{prenom}, \text{numeq}) = (\text{numero}, \text{numeq})$
- ❖ $\neg P1 \cap \neg P3 = (\text{numero}, \text{nom}, \text{prenom}) \cap (\text{numero}, \text{prenom}, \text{numeq}) = (\text{numero}, \text{prenom})$

➔ Fragments verticaux:

- ❖ $F1 = \prod \text{numero}(\text{Cuisinier})$
- ❖ $F2 = \prod \text{numero, nom}(\text{Cuisinier})$
- ❖ $F3 = \prod \text{numero, numeq}(\text{Cuisinier})$
- ❖ $F4 = \prod \text{numero, prenom}(\text{Cuisinier})$

- Allocation des fragments

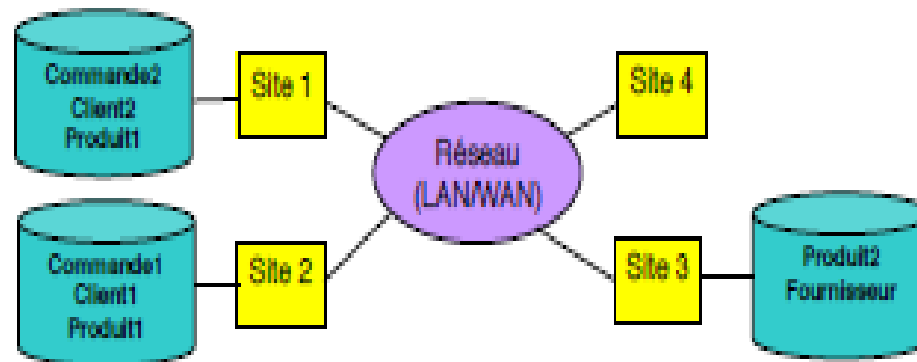
ALLOCATION DE FRAGMENTS AUX SITES

○ Non-Dupliquée :

- chaque fragment n'existe que sur un seul site

○ Dupliquée :

- chaque fragment est sur **plus d'un site**
- maintien de la **cohérence des copies multiples**



PROCESSUS D'ALLOCATION DES FRAGMENTS.

- L'affectation des fragments sur les sites est décidée en fonction de l'origine prévue des requêtes qui ont servi à la fragmentation.
- Le but est de placer les fragments pour minimiser les transferts de données entre les sites.
- L'allocation peut se faire avec **réplication** ou **sans réplication**.
- La **réplication** favorise les performances des requêtes et augmente la disponibilité des données, mais est coûteuse en mise à jour des différents fragments.

PROBLÈME D'ALLOCATION:

○ Entrées:

- □ $F = \{F1, F2, \dots, F_n\}$ - ensemble de fragments
- □ $S = \{S1, S2, \dots, S_m\}$ - ensemble de sites
- □ $Q = \{Q1, Q2, \dots, Q_l\}$ - ensemble de requêtes
- **$\Rightarrow$ Le problème d'allocation consiste à trouver une distribution optimale de F sur S afin d'améliorer la performance (temps de réponse, données transférées, etc.)**

PROBLÈME D'ALLOCATION

- Pour assurer l'optimalité, nous devons respecter les deux conditions:
 - 1. Le coût minimal
 - 2. La performance

PROBLÈME D'ALLOCATION

- 1. Le coût minimal

Les coûts traités dans cette condition sont:

- Le coût de stockage de chaque fragment F_i sur le site S_j ;
- Le coût de consultation de F_i localisé sur S_j ;
- Le coût de mise à jour de F_i sur tous les sites où il est stocké;
- Le coût de communication.

PROBLÈME D'ALLOCATION

○ 2. La performance

Dans cette condition nous essayons de:

- Minimiser le temps de réponse;
- Maximiser le débit de chaque site.

○ Remarque:

○ Suite à la complexité du problème, on ne peut pas satisfaire toutes ces conditions.

○ Les modèles existants essaient de satisfaire les conditions qui les intéressent.

○ Autres contraintes

- o Stockage, équilibrage de la charge entre les sites

ALGORITHMES D'ALLOCATION

- Il existe plusieurs algorithmes, nous présentons ici l'algorithme « **BEST FIT** »
- Principe
 - Permet de déterminer le site sur lequel stocker le fragment, là où l'on maximise les traitements locaux
 - n'intègre pas le problème de la réplication

ALGORITHMES D'ALLOCATION

- Exemple
- Soient les informations suivantes relatives aux différents fragments :

Fragments	temps d'accès local (ms)	temps mise à jour local	temps accès distant	temps m.a.j. distant
R1	100	150	500	600
R2	150	200	650	700
R3	200	250	1000	1100

But: allouer chaque fragment sur le site qui l'invoque le plus

Transactions	sites	fréquence	accès tables (L= lecture, E = écriture)
T1	S1,S4,S5	1	4 accès R1 (3L, 1E) 2 accès R2 (2L)
T2	S2,S4	2	2 accès R1 (2L) 4 accès R3 (3L, 1E)
T3	S3,S5	3	4 accès R2 (3L, 1E) 2 accès R3 (2L)

ALGORITHMES D'ALLOCATION

- Si on récapitule les deux tableaux dans un seul tableau de synthèse on aura :
- Combien de fois un fragment est sollicité par un site lors de toutes les transactions

Fragment	Site	Accès T1 * Fréquence	Accès T2 * Fréquence	Accès T3 * Fréquence	Total pondéré
R1	S1	4*1	0	0	4
	S2	0	2*2	0	4
	S3	0	0	0	0
	S4	4*1	2*2	0	8
	S5	4*1	0	0	4
R2	S1	2*1	0	0	2
	S2	0	0	0	0
	S3	0	0	4*3	12
	S4	2*1	0	0	2
	S5	2*1	0	4*3	14
R3	S1	0	0	0	0
	S2	0	4*2	0	8
	S3	0	0	2*3	6
	S4	0	4*2	0	8
	S5	0	0	2*3	6

ALGORITHMES D'ALLOCATION

- Selon les données de la colonne « Total pondéré » on a :
 - ☐ **On stocke R1 sur S4**
 - ☐ **On stocke R2 sur S5**
 - ☐ **On stocke R3 sur S2 ou S4 (S2 pour mieux répartir)**

- Replication

DÉFINITIONS

- Réplica ou copie de données
 - Fragment horizontal ou vertical d'une table stockée dans une base de données qui est copiée et transféré vers une autre base de données
 - L'original est appelé la copie primaire et les copies sont appelées copies secondaires
- Transparence
 - Les applications clientes croient à l'existence d'une seule copie des données qu'ils manipulent

LES AVANTAGES DE LA RÉPLICATION

- Amélioration des performances
 - lecture de la copie la plus proche
 - évitement du goulot d'étranglement du serveur unique
- Amélioration de la disponibilité
 - lors d'une panne d'un serveur, on peut se replier sur l'autre
 - Disponibilité = $1 - \text{probabilité_panne}^N$
 - probabilité de panne = 5% et 2 copies => disponibilité = 99.75%

DISPONIBILITÉ: VALEURS USUELLES

Taux	Temps d'arrêt par an
90 %	876 heures soit plus de 36 jours
95 %	438 heures soit plus de 18 jours
99 %	87 heures, 36 minutes soit plus de 3 jours et demi
99,9 %	8 heures, 45 minutes, 36 secondes
99,99 %	52 minutes, 33,6 secondes
99,999 %	5 minutes, 15,36 secondes
99,9999 %	31,68 secondes

LES PROBLÈMES DE LA RÉPLICATION

- Convergence

- les copies doivent être maintenues à jour
- à un instant donné, elles peuvent être différentes
- mais elles doivent converger vers un même état cohérent où toutes les mises à jour sont exécutées partout dans le même ordre

- Transparence : le SGBD doit assurer

- la diffusion et la réconciliation des mises à jour
- la résistance aux défaillances

- Règle :

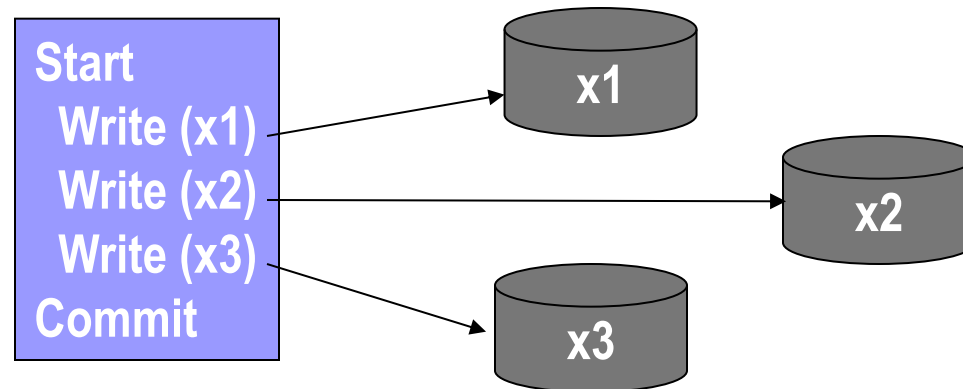
- La réplication est avantageuse s'il y a plus de lectures que d'écritures

OPERATIONS DE REPLICATION

- Définition des objets répliqués
 - table cible = sous-ensemble horizontal et/ou vertical d'une ou plusieurs tables
- Définition de la fréquence de rafraichissement
 - immédiat (après mise à jour des tables primaires)
 - à intervalles réguliers (heure, jour, etc.)
 - à partir d'un événement produit par l'application
- Rafraichissement
 - complet ou partiel (propagation des modifications)
 - push (primaire → secondaires) ou pull (secondaires → primaire)

DIFFUSION SYNCHRONE

- Une transaction met à jour toutes les copies de toutes les données qu'elle modifie.
 - + mise à jour en temps réel des données
 - trop coûteux pour la plupart des applications



DIFFUSION ASYNCHRONE

- Chaque transaction met à jour une seule copie et la mise-à-jour des autres copies est différée (dans d'autres transactions)
 - + mise-à-jour en temps choisi des données
 - l'accès à la dernière version n'est pas garanti